# 8. Robustness and Performance

Once you've written a useful Python program, the next step is to *productionize* your code so it's bullet-proof. Making programs dependable when they encounter unexpected circumstances is just as important as making programs with correct functionality. Python has built-in features and modules that aid in hardening your programs so they are robust in a wide variety of situations.

One dimension of robustness is scalability and performance. When you're implementing Python programs that handle a non-trivial amount of data, you'll often see slowdowns caused by the algorithmic complexity of your code or other types of computational overhead. Luckily, Python includes many of the algorithms and data structures you need to achieve high performance with minimal effort.

## ITEM 65: TAKE ADVANTAGE OF EACH BLOCK IN TRY/EXCEPT/ELSE/FINALLY

There are four distinct times when you might want to take action during exception handling in Python. These are captured in the functionality of `try`, `except`, `else`, and `finally` blocks. Each block serves a unique purpose in the compound statement, and their various combinations are useful (see Item 87: "Define a Root `Exception` to Insulate Callers from APIs" for another example).

### `finally` Blocks

Use `try/finally` when you want exceptions to propagate up but also want to run cleanup code even when exceptions occur. One common usage of `try/finally` is for reliably closing file handles (see Item 66: "Consider `contextlib` and `with` Statements for Reusable `try/finally` Behavior" for another —likely better—approach):

Click here to view code image

```
def try_finally_example(filename):
    print('* Opening file')
handle = open(filename, encoding='utf-8') # Maybe OSError
try:
    print('* Reading data')
    return handle.read()  # Maybe UnicodeDecodeError
finally:
    print('* Calling close()')
    handle.close()        # Always runs after try block
```

Any exception raised by the `read` method will always propagate up to the calling code, but the `close` method of `handle` in the `finally` block will run first:

Click here to view code image

```
filename = 'random_data.txt'

with open(filename, 'wb') as f:
    f.write(b'\xf1\xf2\xf3\xf4\xf5')  # Invalid utf-8

data = try_finally_example(filename)

>>>
* Opening file
* Reading data
* Calling close()
Traceback ...
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xf1 in
➥position 0: invalid continuation byte
```

You must call `open` before the `try` block because exceptions that occur when opening the file (like `OSError` if the file does not exist) should skip the `finally` block entirely:

Click here to view code image

```
try_finally_example('does_not_exist.txt')

>>>
* Opening file
Traceback ...
FileNotFoundError: [Errno 2] No such file or directory:
➥'does_not_exist.txt'
```

## **else** Blocks

Use `try`/`except`/`else` to make it clear which exceptions will be handled by your code and which exceptions will propagate up. When the `try` block doesn't raise an exception, the `else` block runs. The `else` block helps you minimize the amount of code in the `try` block, which is good for isolating potential exception causes and improves readability. For example, say that I want to load JSON dictionary data from a string and return the value of a key it contains:

Click here to view code image

```
import json

def load_json_key(data, key):
    try:
        print('* Loading JSON data')
        result_dict = json.loads(data)  # May raise ValueError
    except ValueError as e:
        print('* Handling ValueError')
        raise KeyError(key) from e
    else:
        print('* Looking up key')
        return result_dict[key]         # May raise KeyError
```

In the successful case, the JSON data is decoded in the `try` block, and then the key lookup occurs in the `else` block:

Click here to view code image

```
assert load_json_key('{"foo": "bar"}', 'foo') == 'bar'

>>>
* Loading JSON data
* Looking up key
```

If the input data isn't valid JSON, then decoding with `json.loads` raises a `ValueError`. The exception is caught by the `except` block and handled:

Click here to view code image

```
load_json_key('{"foo": bad payload', 'foo')

>>>
* Loading JSON data
* Handling ValueError
Traceback ...
JSONDecodeError: Expecting value: line 1 column 9 (char 8)

The above exception was the direct cause of the following
➥exception:

Traceback ...
KeyError: 'foo'
```

If the key lookup raises any exceptions, they propagate up to the caller because they are outside the `try`

block. The `else` clause ensures that what follows the `try/except` is visually distinguished from the `except` block. This makes the exception propagation behavior clear:

Click here to view code image

```
load_json_key('{"foo": "bar"}', 'does not exist')
>>>
* Loading JSON data
* Looking up key
Traceback ...
KeyError: 'does not exist'
```

## Everything Together

Use `try/except/else/finally` when you want to do it all in one compound statement. For example, say that I want to read a description of work to do from a file, process it, and then update the file in-place. Here, the `try` block is used to read the file and process it; the `except` block is used to handle exceptions from the `try` block that are expected; the `else` block is used to update the file in place and allow related exceptions to propagate up; and the `finally` block cleans up the file handle:

Click here to view code image

```
UNDEFINED = object()

def divide_json(path):
    print('* Opening file')
    handle = open(path, 'r+')    # May raise OSError
    try:
        print('* Reading data')
        data = handle.read()     # May raise UnicodeDecodeError
        print('* Loading JSON data')
        op = json.loads(data)    # May raise ValueError
        print('* Performing calculation')
        value = (
            op['numerator'] /
            op['denominator'])   # May raise ZeroDivisionError
    except ZeroDivisionError as e:
        print('* Handling ZeroDivisionError')
        return UNDEFINED
    else:
        print('* Writing calculation')
        op['result'] = value
        result = json.dumps(op)
        handle.seek(0)           # May raise OSError
        handle.write(result)     # May raise OSError
```

```
        return value
    finally:
        print('* Calling close()')
        handle.close()          # Always runs
```

In the successful case, the `try`, `else`, and `finally` blocks run:

Click here to view code image

```
temp_path = 'random_data.json'

with open(temp_path, 'w') as f:
    f.write('{"numerator": 1, "denominator": 10}')

assert divide_json(temp_path) == 0.1

>>>
* Opening file
* Reading data
* Loading JSON data
* Performing calculation
* Writing calculation
* Calling close()
```

If the calculation is invalid, the `try`, `except`, and `finally` blocks run, but the `else` block does not:

Click here to view code image

```
with open(temp_path, 'w') as f:
    f.write('{"numerator": 1, "denominator": 0}')

assert divide_json(temp_path) is UNDEFINED

>>>
* Opening file
* Reading data
* Loading JSON data
* Performing calculation
* Handling ZeroDivisionError
* Calling close()
```

If the JSON data was invalid, the `try` block runs and raises an exception, the `finally` block runs, and then the exception is propagated up to the caller. The `except` and `else` blocks do not run:

Click here to view code image

```
with open(temp_path, 'w') as f:
    f.write('{"numerator": 1 bad data')

divide_json(temp_path)

>>>
* Opening file
* Reading data
* Loading JSON data
* Calling close()
Traceback ...
JSONDecodeError: Expecting ',' delimiter: line 1 column 17
➥(char 16)
```

This layout is especially useful because all of the blocks work together in intuitive ways. For example, here I simulate this by running the `divide_json` function at the same time that my hard drive runs out of disk space:

Click here to view code image

```
with open(temp_path, 'w') as f:
     f.write('{"numerator": 1, "denominator": 10}')

divide_json(temp_path)

>>>
* Opening file
* Reading data
* Loading JSON data
* Performing calculation
* Writing calculation
* Calling close()
Traceback ...
OSError: [Errno 28] No space left on device
```

When the exception was raised in the `else` block while rewriting the result data, the `finally` block still ran and closed the file handle as expected.

## Things to Remember

✦ The `try/finally` compound statement lets you run cleanup code regardless of whether exceptions were raised in the `try` block.

✦ The `else` block helps you minimize the amount of code in `try` blocks and visually distinguish the suc-

cess case from the `try/except` blocks.

✦ An `else` block can be used to perform additional actions after a successful `try` block but before common cleanup in a `finally` block.

## ITEM 66: CONSIDER CONTEXTLIB AND WITH STATEMENTS FOR REUSABLE TRY/FINALLY BEHAVIOR

The `with` statement in Python is used to indicate when code is running in a special context. For example, mutual-exclusion locks (see Item 54: "Use `Lock` to Prevent Data Races in Threads") can be used in `with` statements to indicate that the indented code block runs only while the lock is held:

Click here to view code image

```
from threading import Lock

lock = Lock()

with lock:
    # Do something while maintaining an invariant
    ...
```

The example above is equivalent to this `try/finally` construction because the `Lock` class properly enables the `with` statement (see Item 65: "Take Advantage of Each Block in `try/except/else/finally`" for more about `try/finally`):

Click here to view code image

```
lock.acquire()
try:
    # Do something while maintaining an invariant
    ...
finally:
  lock.release()
```

The `with` statement version of this is better because it eliminates the need to write the repetitive code of the `try/finally` construction, and it ensures that you don't forget to have a corresponding `release` call for every `acquire` call.

It's easy to make your objects and functions work in `with` statements by using the `contextlib` built-in module. This module contains the `contextmanager` decorator (see Item 26: "Define Function

Decorators with `functools.wraps`" for background), which lets a simple function be used in `with` statements. This is much easier than defining a new class with the special methods `__enter__` and `__exit__` (the standard way).

For example, say that I want a region of code to have more debug logging sometimes. Here, I define a function that does logging at two severity levels:

Click here to view code image

```
import logging

def my_function():
    logging.debug('Some debug data')
    logging.error('Error log here')
    logging.debug('More debug data')
```

The default log level for my program is `WARNING`, so only the error message will print to screen when I run the function:

```
my_function()

>>>
Error log here
```

I can elevate the log level of this function temporarily by defining a context manager. This helper function boosts the logging severity level before running the code in the `with` block and reduces the logging severity level afterward:

Click here to view code image

```
from contextlib import contextmanager

@contextmanager
def debug_logging(level):
    logger = logging.getLogger()
    old_level = logger.getEffectiveLevel()
    logger.setLevel(level)
    try:
        yield
    finally:
        logger.setLevel(old_level)
```

The `yield` expression is the point at which the `with` block's contents will execute (see Item 30: "Consider Generators Instead of Returning Lists" for background). Any exceptions that happen in the `with` block will be re-raised by the `yield` expression for you to catch in the helper function (see Item 35: "Avoid Causing State Transitions in Generators with `throw`" for how that works).

Now, I can call the same logging function again but in the `debug_logging` context. This time, all of the debug messages are printed to the screen during the `with` block. The same function running outside the `with` block won't print debug messages:

```
with debug_logging(logging.DEBUG):
    print('* Inside:')
    my_function()

print('* After:')
my_function()

>>>
* Inside:
Some debug data
Error log here
More debug data
* After:
Error log here
```

## Using `with` Targets

The context manager passed to a `with` statement may also return an object. This object is assigned to a local variable in the `as` part of the compound statement. This gives the code running in the `with` block the ability to directly interact with its context.

For example, say I want to write a file and ensure that it's always closed correctly. I can do this by passing `open` to the `with` statement. `open` returns a file handle for the `as` target of `with`, and it closes the handle when the `with` block exits:

Click here to view code image

```
with open('my_output.txt', 'w') as handle:
    handle.write('This is some data!')
```

This approach is more Pythonic than manually opening and closing the file handle every time. It gives you confidence that the file is eventually closed when execution leaves the `with` statement. By highlight-

ing the critical section, it also encourages you to reduce the amount of code that executes while the file handle is open, which is good practice in general.

To enable your own functions to supply values for `as` targets, all you need to do is `yield` a value from your context manager. For example, here I define a context manager to fetch a `Logger` instance, set its level, and then `yield` it `as` the target:

Click here to view code image

```
@contextmanager
def log_level(level, name):
    logger = logging.getLogger(name)
    old_level = logger.getEffectiveLevel()
    logger.setLevel(level)
    try:
        yield logger
    finally:
        logger.setLevel(old_level)
```

Calling logging methods like `debug` on the `as` target produces output because the logging severity level is set low enough in the `with` block on that specific `Logger` instance. Using the `logging` module directly won't print anything because the default logging severity level for the default program logger is `WARNING`:

Click here to view code image

```
with log_level(logging.DEBUG, 'my-log') as logger:
    logger.debug(f'This is a message for {logger.name}!')
    logging.debug('This will not print')

>>>
This is a message for my-log!
```

After the `with` statement exits, calling debug logging methods on the `Logger` named `'my-log'` will not print anything because the default logging severity level has been restored. Error log messages will always print:

Click here to view code image

```
logger = logging.getLogger('my-log')
logger.debug('Debug will not print')
```

```
logger.error('Error will print')

>>>
Error will print
```

Later, I can change the name of the logger I want to use by simply updating the `with` statement. This will point the `Logger` that's the `as` target in the `with` block to a different instance, but I won't have to update any of my other code to match:

Click here to view code image

```
with log_level(logging.DEBUG, 'other-log') as logger:
    logger.debug(f'This is a message for {logger.name}!')
    logging.debug('This will not print')

>>>
This is a message for other-log!
```

This isolation of state and decoupling between creating a context and acting within that context is another benefit of the `with` statement.

## Things to Remember

✦ The `with` statement allows you to reuse logic from `try/finally` blocks and reduce visual noise.

✦ The `contextlib` built-in module provides a `contextmanager` decorator that makes it easy to use your own functions in `with` statements.

✦ The value yielded by context managers is supplied to the `as` part of the `with` statement. It's useful for letting your code directly access the cause of a special context.